

Dynamiske System Mappe 2

Vegard Pettersen

May 22, 2025

Contents

1	KI-deklarasjon	1
2	Introduksjon	2
3	Teori	3
3.1	Ulineært tilstandsrom	3
3.2	Jacobian matrise	3
3.3	Historie	3
4	Modellering	4
4.1	Framgangsmåte	4
4.2	Lineær modell	4
4.3	Utvidet lineær modell	6
4.4	Ulineær modell	8
4.5	Ulineært Jagerfly	11
5	Simulering og regulering	13
5.1	Simulering	13
5.2	Kontrollerbarhet og Observerbarhet	15
5.2.1	Kontrollerbarhet	17
5.2.2	Observerbarhet	18
5.3	Regulering	19
6	Etikk	24
7	Konklusjon	24

1 KI-deklarasjon

KI var brukt i denne oppgaven til enkelte deler. Noen kodesnutter som er klart markert med kommentarer eller nevnt eksplisitt i teksten, var generert av KI. Kode som ikke er markert som KI generert er skrevet av meg. KI var også brukt til å svare på enkelte spesifikke spørsmål som er vanskelig eller umulig å finne konkret svar på fra nettet. For eksempel samtaler som:

- Meg: "Observerbarhet bruker bare A og C matrisen, men i teorien kan også B matrisen brukes til å tilnærme tilstander, for eksempel thrust pådrag kan bidra til å tilnærme fart, posisjon og akselerasjon. Så hvorfor brukes ikke B matrisen for observerbarhet?"
- Forkortet svar fra chatGPT: Observerbarhet handler om tilstander som er mulig å observere kun fra systemdynamikk og eksplisitt målte tilstander. At det ikke er observerbart betyr ikke nødvendigvis at det er umulig å tilnærme fra pådrag.

KI var ikke brukt til å skrive eller strukturere noen deler av teksten, annet enn å gi forslag til forbedringspunkter eller peke på skrivefeil. KI var også brukt som en form for "sanity check" når det kommer til matematikken til systemet, men det var ikke brukt til å generer formler, funksjoner eller parameter.

2 Introduksjon

Systemet jeg skal modellere og simulere er et ustabil 5DOF jagerfly som er kontrollert av fly-by-wire med 5 PID kontrollere, og modellert i et ulineært tilstandsrom. Til å begynne med modellerer og simulerer jeg ett enkelt stabilt fly i lineært tilstandsrom på matrise form, før det blir utvidet til den fulle modellen over flere iterasjoner. På denne måten kan jeg modellere en fullstendig kompleks modell, men likevel teste underveis, slik at modellen er overkommelig å modellere. Modellen vil ikke være modellert etter ett spesifikt fysisk jagerfly, altså jeg skal ikke regne ut luftmotstand, treghetsmoment, løftekraft og slikt med å bruke parameterne til ett ekte eller oppfunnet jagerfly. Isteden vil formel og tilstander utvikles direkte ut ifra ønsket oppførsel til ett jagerfly, så det er mulig at endelig modell er fysisk sett umulig å eksistere i virkeligheten. Modellen vil være modellert generelt etter ett moderne jagerfly, der systemets egen-respons er ustabil. Dette er typisk i moderne jagerfly, ettersom at kontrolloverflatene kan dytte flyet slik at flyets dynamikk fullfører resten av manøveren mye fortare enn i ett stabilt fly. Til gjengjeld betyr det at flyet må reguleres av datasystemer når det skal være stabilt.

Det endelige målet med modellen er å simulere at jagerflyet først letter fra bakken og oppnår steady state flying i luften med konstant fart og altitude, så utfører en [Pugachev Cobra manøver](#), så gjenoppretter kontroll i luften med lavere fart, og til slutt skrur av regulatorer for å falle ukontrollert gjennom luften. Dette målet er valgt siden det vil kreve en relativt presis ulineær modell av flyet, og stiller derfor som en god "stress test" av systemet. I tillegg, siden systemet har mange tilstander, og er derfor vanskelig å visualisere fra bare plot, vil flyets tilstander fra simuleringen eksporteres til ett eksternt program (Unity), der det visualiseres i 3D. Flyets vinkler (yaw, pitch, roll) vil være kontrollert av tre individuelle PID regulatorer, som regulerer vinklene til ett setpoint ved å gi pådrag til kontrolloverflater. Setpoints til disse PID regulatorene er da kontroll av piloten. I tillegg vil flyet kunne sette en mål-altitude og mål-hastighet, og disse vil nås med bruk av to individuelle PID regulatorer, en som kontrollerer hastigheten ved å gi thrust pådrag, og den andre som kontrollerer altituden med å kontrollere setpoint til pitch PID regulatoren. Det er altså en indre reguleringsløyfe (vinkler og hastighet), og en ytre reguleringsløyfe (altitude). Figur 1 viser reguleringsløyfen av det endelige systemet, der det er medregnet at alle tilstander er observerbar fra sensorer uten tilstandsestimatorer.

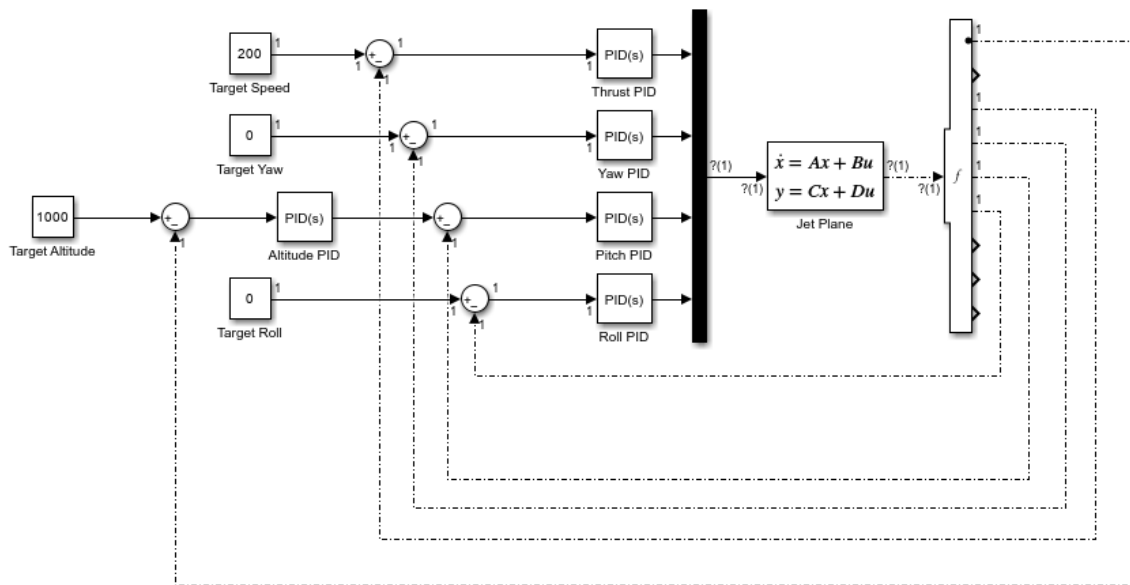


Figure 1: Reguleringsløyfe av endelig MIMO jetfly system

3 Teori

3.1 Ulineært tilstandsrom

Ett ulineært tilstandsrom modelleres som en vilkårlig funksjon av $\dot{x}$. Spesifikt modelleres det som $\dot{x} = f(x, u)$, $y = g(x, u)$ [3]. Til forskjell på lineært tilstandsrom kan ikke et ulineært tilstandsrom direkte regnes observerbarhet og kontrollerbarhet, men det kan dekke ulineær dynamikk, som i enkelte tilfeller er nødvendig for å presist modellere ett system [3]. For å beregne kontrollerbarhet og observerbarhet må det først lineariseres rundt et trim-punkt, og det er kontrollerbarheten og observerbarheten kun for det ene trim-punktet. Derfor må Observerbarhet og kontrollerbarhet testes for flere punkter i det ulineære tilstandsrommet.

3.2 Jacobian matrise

En av metodene for å linearisere ett ulineært tilstandsrom er å finne jacobian matrisen. For en flervariabel vektor funksjon (slik som et ulineært tilstandsrom), er Jacobian matrisen de førsteordens deriverte til alle variablene i funksjonen [4]. Dette kan brukes til å linearisere det ulineære tilstandsrommet, ettersom at en linearisering rundt ett punkt er $a \cdot x + b$, der a er stigningstallet, og b er en konstant. I ett lineært tilstandsrom vil ikke b være nødvendig, ettersom at tilstandsrommet modellerer dynamikken rundt punktet, ikke nødvendigvis en numerisk korrekt linearisering av en ulineær differensialligning. Jacobian matrisen holder altså alle disse stigningstallene, og utregningen av selve tilstandsrommet etter dette innebærer bare å dele opp Jacobian matrisen i en A og B matrise.

3.3 Historie

Utviklingen av jagerfly er i stor grad drevet av nødvendigheten for høy hastighet og manøvrerbarhet [1]. De tidligste jagerflyene var i stor grad like vanlige fly på den tiden, men utstyrt med våpen og med fokus på manøvrerbarhet. Disse tidlige flyene hadde stabil egen-respons, og brukte kontrolloverflater som var direkte drevet av vaiere og trinser fra piloten [1]. Problemet med disse var at de var fysisk tung å kontrollere fra piloten, og at de hadde begrenset manøvrerbarhet og presisjon.

Med høyere fart ble det nødvendig med bedre kontroll-systemer. For eksempel hydrauliske systemer som forbedrer manøvrerbarhet, og reduserer arbeidsinnsats fra piloten [1]. Enda en stor forbedring i kontrollteknologien var Fly-By-Wire teknologi (FBW), der pilotens kontroll kommer fra elektroniske signaler istedenfor mekaniske forbindelser [1]. Altså at ett datasystem tolker kommandoen til piloten, og realiserer det gjennom datastyrte kontrolloverflater [1].

En videre utvikling av Jagerfly var å lage de slik at flyet med vilje har en ustabil egen-respons [2]. En ustabil egen-respons fører til økt manøvrerbarhet, ettersom flyets dynamikk kan støtte kontroll-pådrag istedenfor å kjempe mot det. I ett stabilt fly vil kontroll-pådraget aktivt gi pådrag for endre tilstander i flyet, mens flyets dynamikk aktivt motvirker kontroll-pådraget, men i ett ustabilt fly kan det gi ett lite kontroll-pådrag, fulgt av at flyets dynamikk tar over og fortsetter effekten som kontroll-pådraget startet [2]. Dette fører til kraftig økt manøvrerbarhet, og er kun mulig med datastyrte reguleringssløfer [2].

4 Modellering

4.1 Framgangsmåte

Modellen modelleres gjennom iterative steg, der hvert steg legger ved stadig mer dynamikk, og simuleres og vurderes. På denne måten er det lettere å validere ønskelig oppførsel og gjøre endringer, uten å utvikle en fullstendig matematisk modell som kanskje ikke gir ønskelig respons under simulering. De iterativt utviklede modellene er som følger:

- Lineær modell av fly med stabil egen-respons.
- Utvidet lineær modell med vinkel-akselerasjoner, tyngdekraft og turbulens.
- Den samme modellen implementert som ett ulineært tilstandsrom med numerisk linearisering for kontrollerbarhet- og observerbarhets tester.
- Avansert ulineær modell av jagerfly med ustabil egen-respons og kontrollsløyfer. Observerbarhet og Kontrollerbarhet testet numerisk.
- Utvidet ulineær modell med vertikal hastighets-dynamikk.

4.2 Lineær modell

Først modellerer jeg en enkel modell av et fly direkte i tilstandsrom. For denne første enkle modellen brukes tilstandene

$$x = \begin{bmatrix} h(Altitude) \\ v(Fart) \\ \psi(Yaw) \\ \theta(Pitch) \\ \phi(Roll) \end{bmatrix}$$

og pådrag

$$u = \begin{bmatrix} u_T(Thrust) \\ u_\psi(YawAileron) \\ u_\theta(PitchAileron) \\ u_\phi(RollAileron) \end{bmatrix}$$

Så modelleres simplifiserte ligninger for de forskjellige tilstandene. Først hvor fort flyet øker i høyde, altså løftekraft. Flyet skal ha mer løftekraft basert på hastighet og pitch, men mindre løftekraft basert på høyde og roll. Disse kan modelleres med en lineær formel:

$$\dot{h} = -0.5h + v + 2\theta - 0.5\phi$$

Videre fart, som skal være påvirket av luftmotstand og thrust. Luftmotstand vil øke med hastighet, vinkel på flyet, og vinkel på kontroll-overflatene, og bli redusert med altitude, som kan modelleres med en lineær formel:

$$\dot{v} = 0.1h - 0.5v - 0.3\psi - 0.3\theta + 5u_T - 0.2(u_\psi + u_\theta + u_\phi)$$

Så kan Yaw, Pitch, og Roll modelleres. Yaw og Pitch vil naturlig dempes på grunn av aerodynamikken til flyet, altså at luften treffer halen til flyet og stabiliserer det mot forover-retning. I virkeligheten vil dette også ha en påvirkning på rettingen til flyets hastighet, men siden denne modellen kun modellerer flyets foroverfart, kan dette ignoreres. I tillegg vil dette være litt upresist siden flyets foroverfart ikke vil være direkte framover mens flyet endrer altitude, da foroverfarten egentlig vil være på en vinkel oppover eller nedover, og pitch da heller burde stabiliseres mot denne vinkelen istedenfor 0. Roll derimot vil ikke ha noen slik damping. Dette kan modelleres med formlene:

$$\dot{\psi} = -0.1\psi + u_\psi$$

$$\dot{\theta} = -0.1\theta + u_\theta$$

$$\dot{\phi} = u_\phi$$

Så kan disse formlene omformuleres til matriseform tilstandsrom, altså $\dot{x} = Ax + Bu$.

$$A = \begin{bmatrix} -0.5 & 1 & 0 & 2 & -0.5 \\ 0.1 & -0.5 & -0.3 & -0.3 & 0 \\ 0 & 0 & -0.1 & 0 & 0 \\ 0 & 0 & 0 & -0.1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 5 & -0.2 & -0.2 & -0.2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Ved å simulere denne modellen i python og plote alle relevante tilstander kan jeg teste om tilstandsrom modellen faktisk modellerer et enkelt fly. Figur 2 viser plottene fra denne modellen. Plottene gir mening iforhold til at det er ett fly, men utelukker mye relevant dynamikk; Flyet starter stillestående på banen, så tjener fart og altitude over tid, der den gradvis når en terminalfart ettersom luftmotstanden blir like sterk som motoren, men mangler å modellere vinkelakserelasjon, turbulens og tyngdekraft. I tillegg har flyet en ekstrem pitch (1.5 radianer), som for det første er urealistisk for et fly etter det har stabilisert seg i luften, og er utenfor området der lineariseringen ($\sin(\theta) = \theta$) er presis.

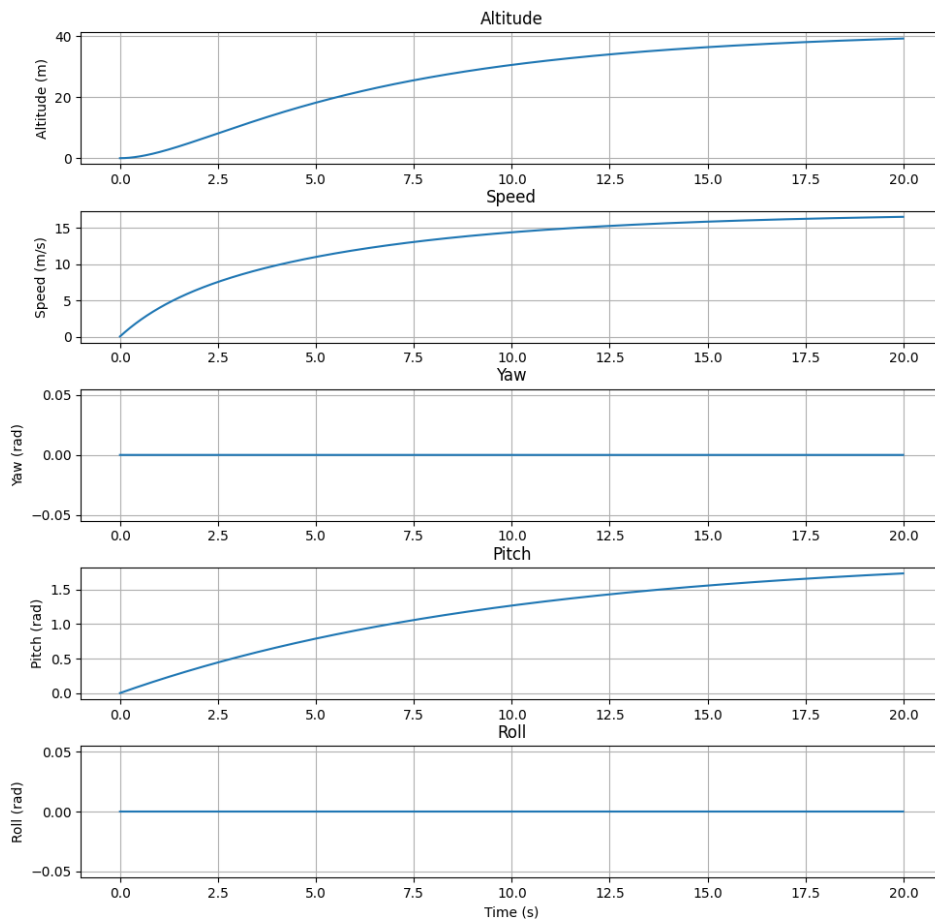


Figure 2: Simulering av enkel tilstandsrom modell

4.3 Utvidet lineær modell

For å inkludere vinkelaskerelasjon, turbulens og tyngdekraft, må det legges til tre nye tilstander og to nye pådrag. Det må altså legges til tilstander: YawRate, PitchRate, og RollRate, og pådragene: turbulens og tyngdekraft. Tyngdekraft pådraget vil alltid være 1 så lenge flyet er på jorda, og turbulens vil være perlin noise over tid. Tilstandsrommet blir da ekspandert til å ha tilstandene:

$$x = \begin{bmatrix} h(Altitude) \\ v(Fart) \\ \psi(Yaw) \\ \theta(Pitch) \\ \phi(Roll) \\ \dot{\psi}(YawRate) \\ \dot{\theta}(PitchRate) \\ \dot{\phi}(RollRate) \end{bmatrix}$$

og pådrag

$$u = \begin{bmatrix} u_T(Thrust) \\ u_\psi(YawAileron) \\ u_\theta(PitchAileron) \\ u_\phi(RollAileron) \\ u_t(Turbulens) \\ u_g(Tyngdekraft) \end{bmatrix}$$

Dette vil også endre det slik at pådraget fra kontroll-overflater setter vinkelakserelasjon istedenfor vinkelfart, så formlene må justeres for dette. I tillegg vil vinkelfarten ha naturlig demping på alle akser fra luftmotstanden. Nye formler for vinklene blir da:

$$\dot{\psi} = -0.1\psi + \dot{\psi}$$

$$\dot{\theta} = -0.1\theta + \dot{\theta}$$

$$\dot{\phi} = \dot{\phi}$$

$$\ddot{\psi} = -0.5\dot{\psi} + u_\psi$$

$$\ddot{\theta} = -0.5\dot{\theta} + u_\theta$$

$$\ddot{\phi} = -0.5\dot{\phi} + u_\phi$$

Ny formel for altitude som inkluderer tyngdekraft blir:

$$\dot{h} = -0.5h + s + 2\theta - 0.5\phi - 9.81u_g$$

Dette kan så realiseres i matrise-form tilstandsrom med matrisene:

$$A = \begin{bmatrix} -0.5 & 1 & 0 & 2 & -0.5 & 0 & 0 & 0 \\ 0.1 & -0.5 & -0.3 & -0.3 & 0 & 0 & 0 & 0 \\ 0 & 0 & -0.1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -0.1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -0.5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -0.5 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -0.5 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & -9.81 \\ 5 & -0.2 & -0.2 & -0.2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Figur 3 viser plot fra alle tilstandene når denne modellen simuleres.

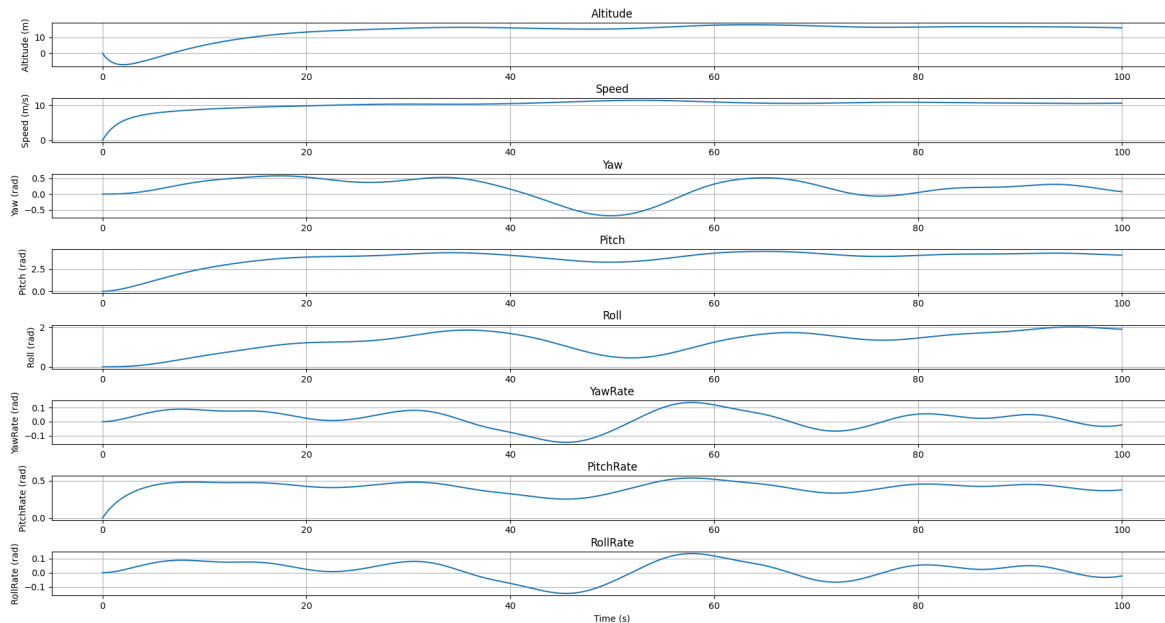


Figure 3: Simulering av enkel flymodell i tilstandsrom med vinkelakserelasjon, turbulens og tyngdekraft

Dette er en mer presis modell av et fly, men det har mange problemer som ikke kan løses med kun tilstandsrom på matriseform. For det første detter flyet gjennom bakken før det får nokk lift til å ta av, istedenfor å kjøre langs bakken før det tar av. Videre modellerer modellen at flyet har mindre løftekraft når det er rullet på en vinkel, men siden det kun kan modelleres lineært, vil flyet faktisk få mer løftekraft om det ruller i negativ retning. I tillegg er det fortsatt mye dynamikk som ikke er inkludert, for eksempel burde løftekraft være nærmere $\sin(\theta)vh$, og det er mulig å linearisere en slik formel for å tilnærme denne dynamikken i steady state, men siden denne modellen også skal modellere at flyet starter fra null fart, høyde og vinkel, så akselerere til å ta av bakken, vil ikke en slik linearisering være presis nokk til å dekke hele simuleringen. Derfor skal modellen som faktisk simuleres være i ulineært tilstandsrom, altså $\dot{x} = f(x, u)$, der $f(x, u)$ er en vilkårlig funksjon som kalkulerer stigningstallet til alle tilstandene i x . En ulineær tilstandsrom modell vil kunne modellere all ønskelig dynamikk, men det kan ikke brukes til å kalkulere observerbarhet eller kontrollerbarhet. I tillegg er en ulineær modell på noen måter enklere, siden effekter som turbulens og tyngdekraft ikke må modelleres med ekstra pådrag, og kan isteden legges til direkte. Derfor vil jeg lage en ulineær modell, så linearisere den numerisk i python for å teste observerbarhet og kontrollerbarhet.

4.4 Ulineær modell

For å lage den ulineære modellen må en formel lages for hver tilstand i tilstandsrommet, men nå kan den være ulineær. For å gjøre implementasjon lettere er tid lagt til som ett ekstra pådrag istedenfor turbulens og tyngdekraft. I tillegg må det også kunne numerisk tilnærmes en vanlig lineær tilstandsrom modell av den ulineære modellen, slik at kontrollerbarhet og observerbarhet kan testes, og dette gjøres ved å finne Jacobian matrisen. Først modelleres samme modell på nytt men i ulineær form, slik at den kan simuleres og sammenlignes med lineær form. For å modellere den ulineære trengs en funksjon $f(x, u)$, som returnerer $\dot{x}$, altså $f(x, u)$, skal være:

$$\begin{aligned}\dot{h} &= -0.5h + s + 2\theta - 0.5\phi - 9.81u_g \\ \dot{v} &= 0.1h - 0.5v - 0.3\psi - 0.3\theta + 5u_T - 0.2(u_\psi + u_\theta + u_\phi) \\ \dot{\psi} &= -0.1\psi + \dot{\psi} \\ \dot{\theta} &= -0.1\theta + \dot{\theta} \\ \dot{\phi} &= \dot{\phi} \\ \ddot{\psi} &= -0.5\dot{\psi} + u_\psi \\ \ddot{\theta} &= -0.5\dot{\theta} + u_\theta \\ \ddot{\phi} &= -0.5\dot{\phi} + u_\phi\end{aligned}$$

I python ser dette slik ut:

```
def f(x, u):
    h, v, yaw, pitch, roll, yawRate, pitchRate, rollRate = x
    thrust, yaw_input, pitch_input, roll_input, t = u

    turbulence = perlinNoise(t)

    hdt = -0.5*h+v+2*pitch-0.5*roll+g
    vdt = 0.1*h-0.5*v-0.3*yaw-0.3*pitch+5*thrust-0.2*(yaw_input+pitch_input+roll_input)+turbulence*0.
    yawdt = -0.1*yaw+yawRate
    pitchdt = -0.1*pitch+pitchRate
    rolldt = rollRate
    yawRatedt = -0.5*yawRate+yaw_input+turbulence
    pitchRatedt = -0.5*pitchRate+pitch_input+turbulence
    rollRatedt = -0.5*rollRate+roll_input+turbulence

    return np.array([hdt,vdt,yawdt,pitchdt,rolldt,yawRatedt,pitchRatedt,rollRatedt])

x = x0.copy()
for t in times:
    x += f(x,u) * dt
    y = x.copy()
```

Og linearisering i python:

```
def compute_jacobian(f, x0, u0, eps=1e-6):
    n = len(x0)
    m = len(u0)
    A = np.zeros((n, n))
    B = np.zeros((n, m))
    f0 = f(x0, u0)

    for i in range(n):
        x_perturb = x0.copy()
        x_perturb[i] += eps
```



```

A[:, i] = (f(x_perturb, u0) - f0) / eps

for i in range(m):
    u_perturb = u0.copy()
    u_perturb[i] += eps
    B[:, i] = (f(x0, u_perturb) - f0) / eps

return A, B

x0 = np.zeros(8)
u0 = np.array([1.0, 0, 0.2, 0, 0])
A, B = compute_jacobian(f, x0, u0)

```

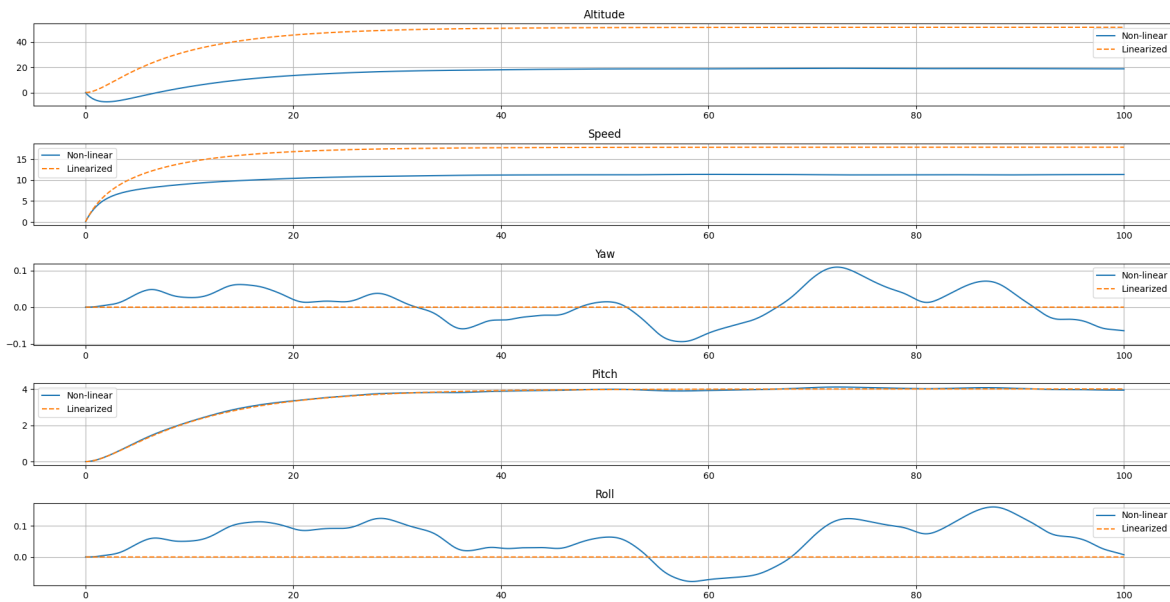


Figure 4: Ulineær og linearisert plot av enkel lineær flymodell i ulineær tilstandsrom form

Figur 4 viser hvordan det ulineære og det lineariserte systemet oppfører seg i samme situasjon som det originale lineære systemet. Figuren viser at systemet oppfører seg på samme måte som det originale lineære systemet, og det lineariserte systemet oppfører seg også som forventet; Det ulineære systemet viser klart samme dynamikk som det originale systemet, men det lineariserte systemet inkluderer ikke tyngdekraft og turbulens, som nå er ulineære funksjoner av t , istedenfor eksterne pådrag. Tyngdekraft og turbulens har ikke noen effekt på obsererbarhet og kontrollerbarhet, så at disse ikke er med i lineariseringen er ikke ett problem. Jacobian-matrisen som python generer er:

```

A =
[[-0.5  1.   0.   2.  -0.5  0.   0.   0. ]
 [ 0.1 -0.5 -0.3 -0.3  0.   0.   0.   0. ]
 [ 0.   0.  -0.1  0.   0.   1.   0.   0. ]
 [ 0.   0.   0.  -0.1  0.   0.   1.   0. ]
 [ 0.   0.   0.   0.   0.   0.   0.   1. ]
 [ 0.   0.   0.   0.   0.  -0.5  0.   0. ]
 [ 0.   0.   0.   0.   0.   0.  -0.5  0. ]
 [ 0.   0.   0.   0.   0.   0.   0.  -0.5]]

B =
[[ 0.   0.   0.   0.   0. ]
 [ 5.  -0.2 -0.2 -0.2  0. ]
 [ 0.   0.   0.   0.   0. ]
 [ 0.   0.   0.   0.   0. ]

```

```
[ 0.  0.  0.  0.  0. ]  
[ 0.  1.  0.  0.  0. ]  
[ 0.  0.  1.  0.  0. ]  
[ 0.  0.  0.  1.  0. ]]
```

Denne jacobian matrisen er nøyaktig samme som den originale matrise form tilstandsrom modellen med unntak av tyngdekraft og turbulens, som er forventet.

4.5 Ulineært Jagerfly

For det nye ulineære systemet skal jeg modellere ett fly som er mer likt et jagerfly enn et vanlig fly, som betyr at flyet med vilje er ustabilt, og kun stabiliseres av aktive datasystemer. For denne modellen skal flyet ha negativ damping i pitch retning, som betyr at en forstyrrelse i pitch som dytter den unna null, vil fortsette å gro ukontrollert, men siden denne modellen er ulineær, og dette modelleres med $\sin(\theta)$, burde flyet forsøke å stabilisere bakover i pitch. Yaw, vil ha naturlig damping, ettersom at halen treffer vinden for å peke flyet i retningen til hastigheten, men hvis flyet snur 180 grader rundt pitch, vil halen peke framover, og derfor vil den stabiliserende effekten bli invertert, slik at flyet forsøker å peke framover igjen i yaw aksen. Dette vil modelleres med å multiplisere den naturlige dampingen i yaw med $\sin(\theta)$, slik at ettersom at pitch øker vil yaw stabiliseringen gradvis bli svakere fram til den blir invertert. Det kan også noteres at når pitch er 90 grader positivt eller negativt vil yaw og roll ligge på samme akse, altså gimbal lock, men dette vil ikke tas hensyn til i denne modellen. Når flyet peker oppover, altså pitch er 90 grader, vil roll ha en naturlig damping mot ± 90 grader, ettersom at det er "path of least resistance" for luften, altså roll vil ha damping proporsjonalt med $\cos(\text{roll}) \cdot |\cos(\text{pitch})|$. Forventet oppførsel for flyet i ukontrollert fall med turbulens, er altså at flyet vil først øke eller minke i pitch mot at pitch er 180 grader, så gjøre det samme i yaw retning for å peke flyet framover igjen, og så repetere. Samtidig som dette vil den beholde vinkelhastighet i begge aksene, og flyet vil da oscillerende rotere ukontrollert på pitch og yaw aksene.

Løftekraften vil være proporsjonal med sinusen til pitch, absoluttverdien til cosinusen til roll, og hastighet i andre. Dette modellerer da at flyet vil ha mer løftekraft med mer fart, mer løftekraft ettersom flyet peker mer oppover, og mindre løftekraft ettersom flyet snur seg sidelengs i roll aksen. I tillegg vil løftekraften bli redusert av tyngdekraften, og eksponentialt redusert av altitude. Ettersom at dette er en simplifisert modell av et fly modellert direkte i tilstandsrom etter ønsket oppførsel, er det ikke nødvendig å beregne faktisk løftekraft fra parameteren til ett fly, og kan isteden modelleres med konstanter. Formelen for løftekraften blir da

$$\dot{h} = \frac{k \cdot v^2 \cdot (\sin(\theta) + 0.1) \cdot |\cos(\phi)|}{h^2 \cdot c + 1} - g$$

I delen $(\sin(\theta) + 0.1)$ er 0.1 lagt til, slik at flyet har litt løftekraft selv om det ikke har noen pitch. For akselerasjon, er det hovedsakelig 2 faktorer, kraft fra motoren, og luftmotstand. Luftmotstanden vil være basert på både vinkelen til kontroll-overflater, og vinkelen til selve flyet, i tillegg til en konstant luftmotstand, proporsjonalt med hastigheten i andre. For denne simplifiserte modellen bruker jeg summen av absoluttverdien til sinusen til alle vinklene pluss en konstant, ganget med hastighet i andre. På denne måten har flyet en luftmotstand i utgangspunktet, som blir større ettersom flyet har mer overflate som møter vinden. I tillegg vil det være mindre luft høyere oppe, så luftmotstanden er også eksponentialt proporsjonal inversen til alitutden. I tillegg bør kraften fra motoren være proporsjonal med cosinusen til pitch og yaw, siden motoren ikke peker framover når flyet peker 90 grader oppover eller til siden. Formelen for akselerasjon blir da

$$\dot{v} = \frac{u_v \cdot \sin(\theta) \cdot \sin(\psi) - (v^2 \cdot d \cdot (|\sin(\theta)| + |\sin(\phi)| + |\sin(\psi)| + |\sin(u_\theta)| + |\sin(u_\phi)| + |\sin(u_\psi)| + 3))}{(h^2 \cdot c + 1)}$$

Så til slutt alle vinkelhastigheter og vinkelakserelasjoner. Pitch vil ha negativ damping proporsjonalt med sinusen til pitch ganget med cosinusen til yaw (slik at flyet vil prøve å holde halen framover i pitch aksen selv om flyet flipper framover i yaw aksen). Yaw vil ha naturlig damping proporsjonalt med sinusen til yaw og cosinusen til pitch (slik at flyet vil prøve å holde halen bakover i yaw aksen selv om flyet flipper i pitch aksen). Roll vil ha damping mot ± 90 grader når pitch er 90, altså det er proporsjonalt med cosinusen til roll og absoluttverdien til cosinusen til pitch. I tillegg må alle disse pådragene fra luften være proporsjonal med hastighet i andre. Disse dampingene blir lagt til som pådrag på vinkelhastighetene. I tillegg vil alle vinkelhastigheter ha naturlig damping fra luftmotstand, i tillegg til turbulens, som vil være implementert som perlin noise. Til slutt er også vinkelakserelasjonen påvirket av kontrolloverflater proporsjonalt med sinusen til vinkelen deres. Formlene blir da:

$$\dot{\psi} = \dot{\psi}$$

$$\dot{\theta} = \dot{\theta}$$

$$\dot{\phi} = \dot{\phi}$$

$$\ddot{\psi} = (-0.25 \cdot \cos(\theta) \cdot \sin(\psi) - 0.25\dot{\psi} + \sin(u_\psi) + \textit{turbulens}) \cdot v^2 \cdot d_2$$

$$\ddot{\theta} = (0.25 \cdot \sin(\theta) - 0.25\dot{\theta} + \sin(u_\theta) + \textit{turbulens}) \cdot v^2 \cdot d_2$$

$$\ddot{\phi} = (-0.05 \cdot |\cos(\theta)| \cdot \cos(\phi) - 0.25\dot{\phi} + \sin(u_\phi) + \textit{turbulens}) \cdot v^2 \cdot d_2$$

Modellen har da fire parameter:

- c : atmosfære tetthet koeffisient
- k : løftekraft koeffisient
- d : luftmotstand koeffisient
- d_2 : vinkel demping koeffisient

5 Simulering og regulering

5.1 Simulering

Videre skal systemet reguleres og simuleres i python. For å få systemet til å oppføre seg som ønsket, har parameterverdiene blitt testet iterativt, fram til plot var som forventet. Jeg har da kommet fram til etter testing at følgende parameterverdier fører til at flyet oppfører seg som forventet i ulike situasjoner.

- k : 50
- c : 20000
- d : 30
- d_2 : $1/1000$

For simuleringen er det også implementert flere elementer som går utenfor det ulineære tilstandsrommet, og gjør simuleringen mer presis. Disse kunne også vært lagt inn som ekstra tilstander, men det er unødvendig. Det første er at vinkler er normalisert til å være mellom $-\pi$ og π , for å gjøre plottene klarere. I tillegg til dette er det lagt inn en funksjon som begrenser altituden og hastigheten til å være minimum 0, slik at flyet ikke kan gå igjennom bakken eller fly bakover. Til slutt er det også lagt til grenser for kontrolloverflatene, slik at de ikke kan roteres mer enn 45 grader. For testing er det lagt inn aggressive P-regulatorer, som stabiliserer yaw, pitch, og roll mot et mål-punkt. I de første 200 sekundene prøver flyet å nå en pitch på 0.2, så mellom 200 og 300 sekund går det gradvis mot en pitch på 0. Etter 400 sekunder skrus av alle regulatorer, og flyet fortsetter å simulere i 200 sekund ukontrollert i luften.

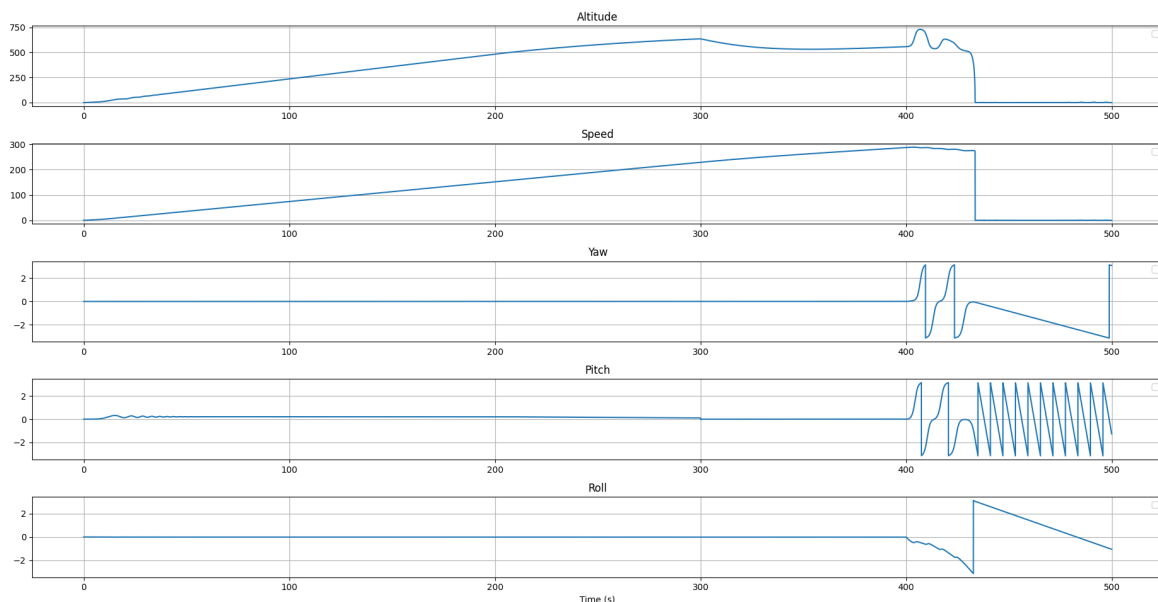


Figure 5: Simulering av ulineær jetfly-modell

Figur 5 viser simuleringen av dette systemet. Oppførselen er som forventet; turbulens har i stor grad blitt regulert ut av de aggressive regulatorene. Flyet er på bakken i en liten stund før det har nokk fart og dermed løftekraft til å overkomme tyngdekraften. Når pitchen flater ut rundt 300 sekund begynner jetflyet å miste altitude, siden modellen av jetflyet er laget slik at det er minimalt med løftekraft når det er lite pitch. Til slutt, når kontrollerne blir skrudd av, mister flyet først kontroll i pitch, ettersom det har negativ damping, fulgt av at yaw mister kontroll siden halen nå peker framover. Samtidig som dette skjer beholder flyet en stor grad av vinkelhastigheten, og dermed tar til slutt en av aksene over (pitch i dette tilfellet), og snurrer alene på den aksen slik at de andre aksene blir til en hvis grad stabile. På grunn av denne ukontrollerte snurringen mister også flyet fort all fart og altitude fra luftmotstand og mangel på løftekraft. Det kan også merkes at flyet faller veldig fort mot bakken,

og mens luftmotstand og løftekraft parametere kan finjusteres for at denne effekten forsvinner, er det lettere å redusere tyngdekraften; Siden dette ikke er en modell av ett ekte jetfly, er det ingen reel betydning for at tyngdekraften skal være -9.81, og denne verdien kan derfor endres. Den er derfor endret til -2.5.

I tillegg til plot er tilstander over tid eksportert til en json fil, som videre er importert til Unity, der tilstandene blir realisert på en modell av et fly. Dette er kun for å enklere visualisere flyet, og koden for denne visualiseringen er nesten fullstendig skrevet av KI, og Unity prosjektet er derfor ikke inkludert som ett vedlegg. Video fra simuleringen i figur 5 er lagt ved som "JetSim1Visu.mp4".

5.2 Kontrollerbarhet og Observerbarhet

Videre er å finne observerbarhet og kontrollerbarhet. Som nevnt tidligere kan det ulineære tilstandsrommet lineariseres med å regne ut jacobian matrisen numerisk. Nå som modellen faktisk inneholder ulineære funksjoner er det ikke lenger nokk å regne ut jacobian matrisen en gang, og teste observerbarhet og kontrollerbarhet; Nå må funksjonen lineariseres på flere punkter, og observerbarhet og kontrollerbarhet må testes for hvert punkt. Dette er annerledes enn observerbarhet og kontrollerbarhet på lineære tilstandsrom, der observerbarhet og kontrollerbarhet regnes ut en gang for hele tilstandsrommet. kontrollerbarhet og observerbarhets kriterier er også annerledes i ulineære tilstandsrom; I lineære tilstandsrom er kontrollerbarhet og observerbarhets kriterier krav for parameter verdier for at rommet skal være observerbart eller kontrollerbart. For eksempel hvis tyngdekraften må være mindre enn null for at rommet skal være kontrollerbart. Mens slike kriterier fortsatt kan eksistere for et ulineært tilstandsrom, er det også kriterier for selve tilstandene for at rommet skal være kontrollerbart. Hver linearisering er altså ett nytt lineært tilstandsrom for ett sett med nåværende tilstander i det ulineære rommet. For eksempel, I denne modellen er ett kontrollerbarhets-kriterie at både yaw og pitch må være noe annet enn 90 grader, fordi når yaw eller pitch er 90 grader, peker ikke jetmotoren fremover, og flyet kan dermed ikke påvirke hastigheten. I praksis er dette ikke ett problem, siden flyet vil aldri være stabilt på 90 grader på enten pitch eller yaw uten at flyet kan rettes ut, og når yaw eller pitch er 90 grader, har flyet større problemer enn kontrollerbarhet. I tillegg, siden dette nå er en numerisk linearisering, må ikke alltid kriteriet være nøyaktig oppfylt, men bare nærme nokk til at numeriske avvik gjør det ukontrollerbart eller uobserverbart. Samme funksjon for å regne Jacobian matrisen som var brukt tidligere er også brukt nå, men nå er det utvidet til en funksjon "LinearizeAndTest(x0,u0)", som tester kontrollerbarhet og observerbarhet gitt x0 og u0, og hvis det er ikke er kontrollerbart eller observerbart, printer det informasjon som kan hjelpe å identifisere kriteriet. Kode for denne funksjonen er:

```
def LinearizeAndTest(x0, u0):
    if not np.all(np.isfinite(x0)) or not np.all(np.isfinite(u0)):
        print("Ikke-lineær simulering gikk ut av kontroll | NaN eller inf i x0/u0.")
        return

    # Lineariser rundt (x0, u0)
    A, B = compute_jacobian(f, x0.copy(), u0.copy())
    n = A.shape[0]
    C = np.eye(n) # Full observasjon

    # === KONTROLLERBARHET ===
    ctrl_mat = B
    for i in range(1, n):
        ctrl_mat = np.hstack((ctrl_mat, np.linalg.matrix_power(A, i) @ B))
    rank_C = np.linalg.matrix_rank(ctrl_mat)

    # SVD og nullroms-vektor
    _, _, Vt = np.linalg.svd(ctrl_mat, full_matrices=False)
    v = Vt.T[:, -1] # minst signifikante retning
    influence = np.abs(ctrl_mat @ v) # hvilke tilstander påvirkes minst
    idx = np.argsort(-influence)[:n - rank_C] # ukontrollerbare

    if rank_C < n:
        print(f"\nKontrollerbarhetsmatrise-rang: {rank_C} / {n}")
        print(f"Ukontrollerbare tilstandsindekser: {idx.tolist()}")
        print(f"x0 = {x0}")
        print(f"Systemet er IKKE fullt kontrollerbart ved t = {t}.\n")

    # === OBSERVERBARHET ===
    obsv_mat = C
    for i in range(1, n):
        obsv_mat = np.vstack((obsv_mat, C @ np.linalg.matrix_power(A, i)))
```

```

rank_0 = np.linalg.matrix_rank(obsv_mat)

# SVD og nullroms-vektor for observabilitet
_, _, Vto = np.linalg.svd(obsv_mat, full_matrices=False)
v_obs = Vto.T[:, -1]
influence_obs = np.abs(obsv_mat @ v_obs)
idx_o = np.argsort(-influence_obs)[:n - rank_0]

if rank_0 < n:
    print(f"\nObserverbarhetsmatrise-rang: {rank_0} / {n}")
    print(f"Uobserverbare tilstandsindekser: {idx_o.tolist()}")
    print(f"x0 = {x0}")
    print(f"Systemet er IKKE fullt observerbart ved t = {t}.\n")

```

Denne funksjonen blir kjørt for hvert simulerte sekund. Det er også mulig å teste dette med grid search, altså å teste alle verdier i et diskret fler-dimensjonalt rom, men det vil også teste mange verdier som ikke er realistisk oppnåelig, og mange kriterier for ukontrollerbarhet og uobserverbarhet kan være akseptable hvis det er usannsynlig eller umulig at tilstanden er vedvaring oppfylt.

5.2.1 Kontrollerbarhet

I testen var flyet ukontrollerbart i noen få tilstander. Den første tilstanden var helt på starten, og det printet:

```
Kontrollerbarhetsmatrise-rang: 7 / 8
Ukontrollerbare tilstandsindekser: [6]
x0 = [0. 0.01 0. 0. 0. 0. 0. 0. ]
Systemet er IKKE fullt kontrollerbart ved t = 0.0.
```

Her er tilstand 6, altså pitchRate ukontrollerbar. Formelen for dette er:

$$\ddot{\theta} = (0.25 \cdot \sin(\theta) - 0.25\dot{\theta} + \sin(u_\theta) + \textit{turbulens}) \cdot v^2 \cdot d_2$$

Ved å se på formelen vil ikke pitch pådraget ha noen påvirkning hvis v er 0. I denne tilstanden er ikke v 0, men den er veldig nært 0, og det er mulig at den dukker opp som ukontrollerbar grunnet numeriske feil. Uansett er det forventet at både pitch, roll, og yaw skal være nesten ukontrollerbar her, ettersom at flyet ikke beveger seg. Det gir også mening logisk, siden kontrolloverflater bruker aerodynamikk for å kontrollere flyet. Videre er det ingen ukontrollerbare tilstander mellom 0 og 488 sekund, altså det er ingen ukontrollerbare tilstander så lenge regulatorene er aktiv. Resten av de ukontrollerbare tilstandene er med 0 fart, med unntak av følgende tilstand:

```
Kontrollerbarhetsmatrise-rang: 2 / 8
Ukontrollerbare tilstandsindekser: [7, 4, 5, 2, 0, 3]
x0 = [ 0.43877151  0.05372095  0.10462975  1.22223988 -2.64693838 -0.07361383
      -0.95120527 -0.05348051]
Systemet er IKKE fullt kontrollerbart ved t = 491.0.
```

Dette er også med lav fart og altitude, og mange ekstreme vinkler, og dermed er det også mulig at det er ukontrollerbart på grunn av numerisk feil her. Fra dette kan vi konkludere at flyet er kontrollerbart.

5.2.2 Obsererbarhet

For å teste observerbarhet må det først bestemmes hvilken sensorer flyet har. Sensorer som flyet har er:

- Barometer: Observerer høyde (h)
- Pitotrør: Observer luftfart (v)
- Gyroskop: Observer vinkelhastigheter ($\dot{\psi}, \dot{\theta}, \dot{\phi}$)
- Akselerometer: Inkludert i IMU med gyroskop, kan interpolere fart og høyde (h, v)
- Kompass: Kan observere absolutt yaw (ψ)
- Inklinometer / kunstig horisont: Observerer absolutt pitch og roll (ϕ, θ)
- GPS: Observerer absolutt posisjon, kan brukes til å regne fart (v)

Her er alle tilstander i tilstandsrommet direkte observert av sensorer, og C matrisen er dermed en identitetmatrise. Det betyr at alle tilstander alltid vil være observerbar til og med uten observatordesign, og det er derfor unødvendig å teste observerbarhet. Observatordesign kan likevel være nødvendig, for eksempel hvis Inklinometeret har betraktning lavere oppdateringsfrekvens enn gyroskopet, kan en observator utvikles for at gyroskopet interpolerer vinkelen hurtig, mens Inklinometeret bekrefter vinkelen på en lavere frekvens, men observatorer vil ikke utvikles i denne modellen.

5.3 Regulering

Til slutt gjenstår å regulere systemet. For å lage en regulator lages først en funksjon $u = h(x)$, altså en funksjon som tar imot tilstandene x , og returnerer pådraget u . I python er dette implementert slik:

```
def h(x, t):
    h, v, yaw, pitch, roll, yawRate, pitchRate, rollRate = x
    thrust = 0
    yawInput = 0
    pitchInput = 0
    rollInput = 0
    return np.array([thrust, yawInput, pitchInput, rollInput, t])
```

Simulate nonlinear system

```
for i in range(steps):
    x += f(x, h(x,t)) * dt
```

Til å begynne med bruker jeg bare en regulator som skal holde yaw, pitch, og roll beint med bruk av pid, og gi konstant maks thrust. For dette lager jeg en PID klasse i python:

```
class PID:
    def __init__(self, kp=1.0, ki=0.0, kd=0.0):
        self.kp = kp
        self.ki = ki
        self.kd = kd

        self.setpoint = 0.0
        self.integral = 0.0
        self.previous_error = 0.0

    def update(self, measurement, dt):
        error = self.setpoint - measurement
        self.integral += error * dt
        derivative = (error - self.previous_error) / dt if dt > 0 else 0.0

        output = (self.kp * error) + (self.ki * self.integral) + (self.kd * derivative)
        self.previous_error = error

        return output
```

og bruker den i pådragsfunksjonen med python koden:

```
yawPID = PID(1,0,0)
pitchPID = PID(1,0,0)
rollPID = PID(1,0,0)
```

Regulator function

```
def h(x, t):
    h, v, yaw, pitch, roll, yawRate, pitchRate, rollRate = x
    thrust = 1
    yawInput = yawPID.update(yaw, dt)
    pitchInput = pitchPID.update(pitch, dt)
    rollInput = rollPID.update(roll, dt)

    return np.array([thrust, yawInput, pitchInput, rollInput, t])
```

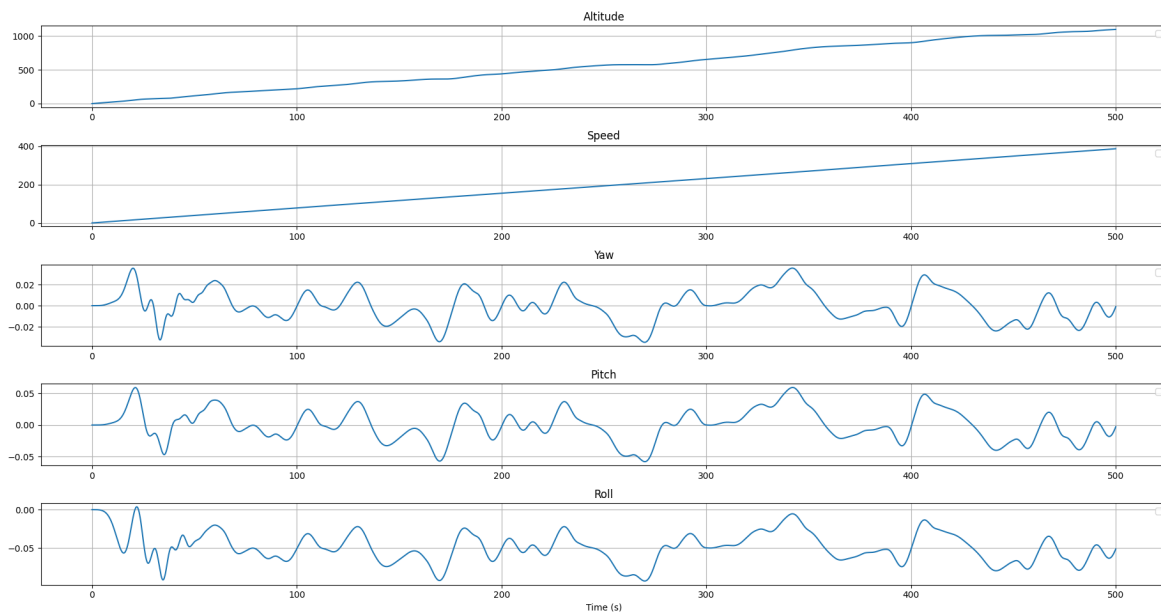


Figure 6: Simulering med PID reguleringsløyper på vinkler og konstant thrust

Figur 6 viser plottet fra denne simulasjonen. Som forventet stabiliserer den vinklene, og tjener fart fram til den tar av. Videre kan vi kontrollere farten med enda en PID regulator. For eksempel en mål-fart på $200 \frac{m}{s}$. Dette implementeres i python slik:

```

yawPID = PID(1,0,0)
pitchPID = PID(1,0,0)
rollPID = PID(1,0,0)
thrustPID = PID(1,0,0)
thrustPID.setpoint = 200

# Regulator function
def h(x, t):
    h, v, yaw, pitch, roll, yawRate, pitchRate, rollRate = x
    thrust = thrustPID.update(v)
    yawInput = yawPID.update(yaw)
    pitchInput = pitchPID.update(pitch)
    rollInput = rollPID.update(roll)

    return np.array([thrust,yawInput,pitchInput,rollInput,t])

```

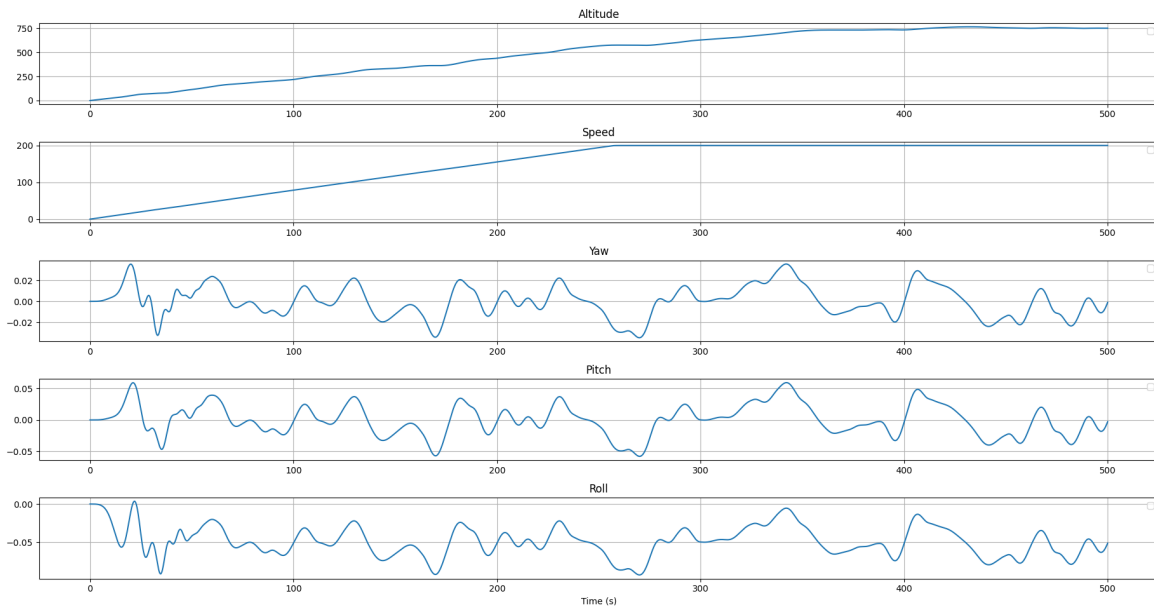


Figure 7: Simulering med mål-fart 200, regulert av PID

Figur 7 viser som forventet at flyet akselerer opp til $200 \frac{m}{s}$, så stabiliserer på det på $200 \frac{m}{s}$. Videre kan det reguleres for å også ha en mål altitude. Dette er mer komplisert, siden det ikke er et direkte pådrag som kontrollerer altitude, og tilstandene som gjør det påvirker hverandre i stor grad. For eksempel, å øke altitude reduserer luftmotstand, som øker fart, som igjen øker altitude. Eller for eksempel at vinkel øker løftkraft, men øker også luftmotstand, som minker fart, som minker luftmotstand. For å kontrollere altitude kontrolleres pitch setpoint med en annen PID regulator, som også begrenser pitch vinkelen. Dette implementeres slik i python:

```

yawPID = PID(1,0,0)
pitchPID = PID(1,0,0)
rollPID = PID(1,0,0)
thrustPID = PID(1,0,0)
thrustPID.setpoint = 200

altitudePID = PID(0.1,0,0) # Ny
altitudePID.setpoint = 1000 # Ny

# Regulator function
def h(x, t):
    h, v, yaw, pitch, roll, yawRate, pitchRate, rollRate = x
    thrust = thrustPID.update(v)
    yawInput = yawPID.update(yaw)

    pitchPID.setpoint = altitudePID.update(h) # Ny
    pitchPID.setpoint = clamp(pitchPID.setpoint,-0.5,0.2) # Ny

    pitchInput = pitchPID.update(pitch)
    rollInput = rollPID.update(roll)

    return np.array([thrust,yawInput,pitchInput,rollInput,t])

```

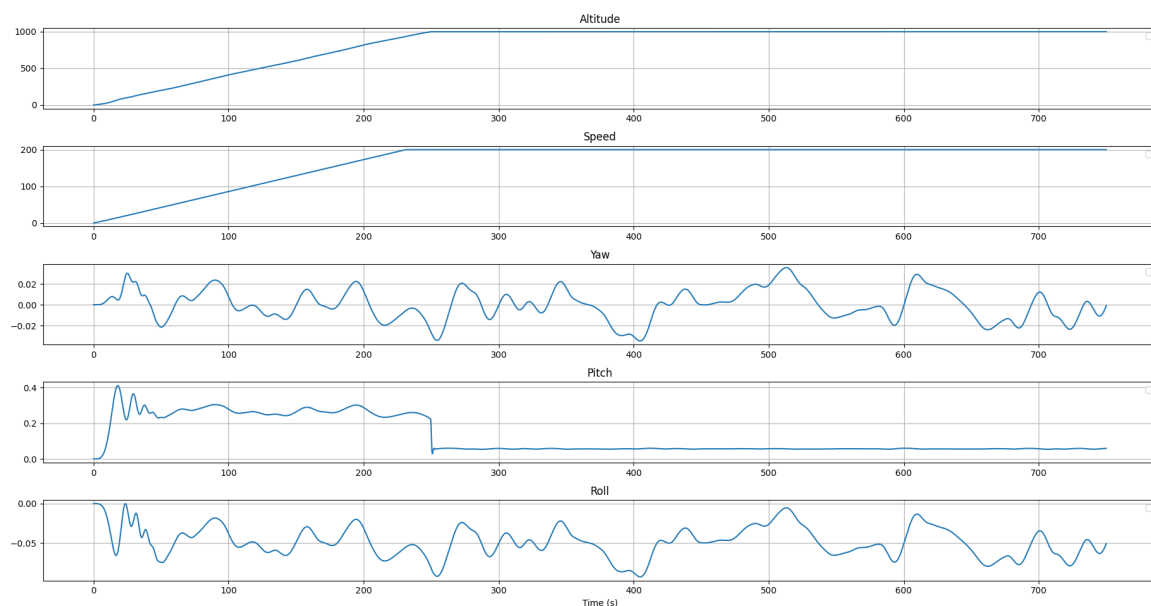


Figure 8: Simulering med mål-altitude kontrollert av PID som driver pitch setpoint

Som forventet, i figur 8, når flyet riktig mål-altitude og målhastighet, og stabiliserer der.

Videre kan en mer interessant manøver simuleres, nemlig Pugachevs Cobra manøveren. Et eksempel på dette vises klart i [dette](#) klippet fra filmen "Top Gun Maverick (2022)", og mens dette klippet er ganske overdrevet, er det mulig å utføre for ekte jagerfly også. For å simulere dette i systemet settes pitch PID setpoint til 90 grader, og thrust PID setpoint til 100 mellom 300 og 400 sekund, altså etter flyet har nådd steady state. I python er det gjort slik:

```
if(t > 300 and t < 400):
    thrustPID.setpoint = 100
    pitchPID.setpoint = np.pi/2
```

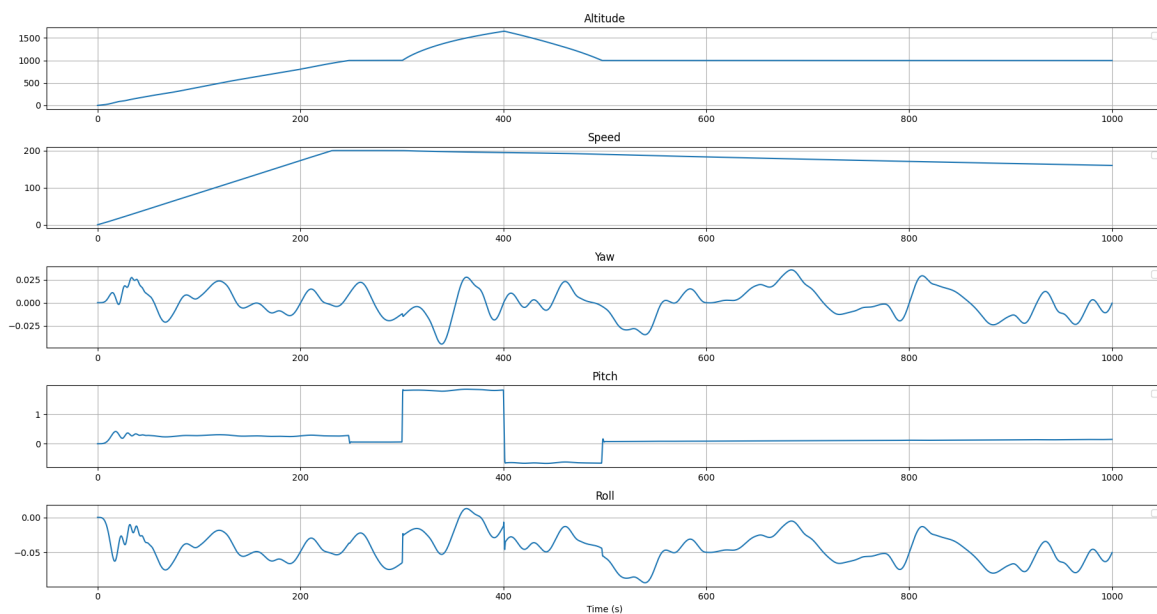


Figure 9: Første simulering av Pugachevs Cobra manøveren

Figur 9 viser plot fra dette, og dette er ikke som forventet. For det første har flyet veldig mye løftekraft når det peker oppover, og for det andre skaper vingene nesten ingen luftmotstand. Dette

skyldes av tre problemer med modellen. For det første modelleres løftekraft i forhold til pitch som $\sin(\theta)$, men dette er feil siden dette vil gi maks løftekraft ved $\theta = \frac{\pi}{2}$, og fortsatt gi løftekraft etter 90 grader. Dette burde egentlig være $\sin(2\theta)$, ettersom at denne funksjonen vil gi null løftekraft ved 90 grader, maks løftekraft ved 45 grader, og negativ løftekraft over 90 grader, som er mer i tråd med oppførselen til et ekte fly. Videre er luftmotstanden modellert som en kombinasjon av absoluttverdien til sinusen til yaw, pitch, roll, og yaw-, pitch-, roll-kontrolloverflater. Dette er upresist ettersom at kontrolloverflatene har like stor påvirkning som hele vingene til flyet, og roll påvirker direkte luftmotstand. For å fikse dette kan vinkelen til selve flyet påvirke luftmotstand 5 ganger mer enn kontrolloverflater, og roll kan fjernes fra funksjonen. Videre er at løftekraft og tyngdekraft er modellert direkte som løftehastighet, mens disse egentlig burde være løfteakserelasjon, slik at flyet kan tjene opp vertikal fart fra løftekraften, og så beholde den selv når løftekraften forsvinner. I tillegg vil dette modellere tyngdekraft mer presist, og det kan legges til vertikal luftmotstand, og thrust-vectoring. For dette må det legges til en ny tilstand for vertikal hastighet. Vertikal luftmotstand kan også nå modelleres, og det modelleres med en simplifisert modell $-\dot{h} \cdot v^2 \cdot 0.0001$ Altså nye formler blir:

$$\begin{aligned}\dot{h} &= \dot{h} \\ \ddot{h} &= \frac{k \cdot v^2 \cdot (\sin(2\theta) + 0.1) \cdot |\cos(\phi)|}{h^2 \cdot c + 1} - g + \sin(\theta) \cdot u_v - \dot{h} \cdot v^2 \cdot 0.01 \\ \dot{v} &= \frac{u_v \cdot \sin(\theta) \cdot \sin(\psi) - (v^2 \cdot d \cdot (5(|\sin(\theta)| + |\sin(\psi)|) + |\sin(u_\theta)| + |\sin(u_\phi)| + |\sin(u_\psi)| + 3))}{h^2 \cdot c + 1}\end{aligned}$$

Siden vertikal hastighet er observerbart fra akselergometer, og altitude er kontrollerbart fra vertikal fart, og vertikal fart er kontrollerbart av det samme som altitude var kontrollert av, er det unødvendig å teste observerbarhet og kontrollerbarhet på nytt. Med denne nye modellen kan manøveren testes på nytt, men siden dynamikken er endret trengs også nye parameter verdier for å stabilisere simulasjonen. Etter testing fører følgende parameter verdier til ønsket oppførsel:

- c : 50
- k : 1000000
- d : 250
- d_2 : $\frac{1}{1000}$

I tillegg, siden simulatoren nå er litt annerledes, er manøveren mellom $t = 500$ og $t = 600$, og etter $t = 800$ er regulatorer skrudd av for å teste ukontrollert dynamikk på nytt.

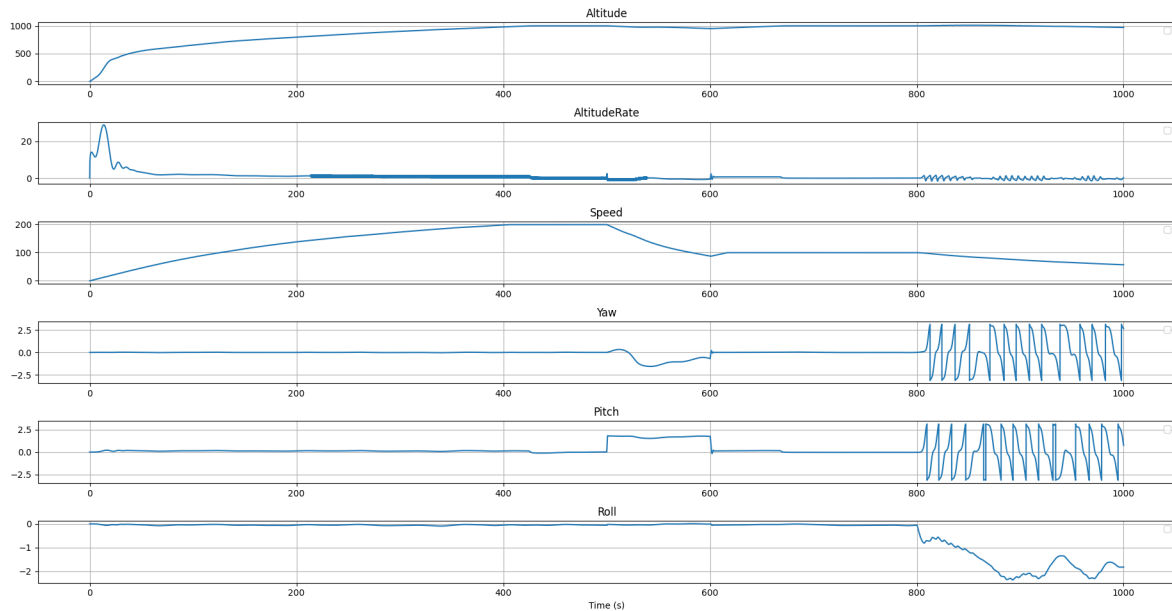


Figure 10: Simulering av Pugachevs Cobra manøveren med forbedret modell

Figur 10 viser den nye simuleringen med utvidet modell, og plottet er mye mer som forventet nå. Ukontrollert dynamikk er også i stor grad som forventet, men flyet mister altitude litt saktere enn forventet. Dette er sannsynligvis på grunn av den simplifiserte modellen for vertikal luftmotstand, som ignorerer vinkel og altitude. I tillegg har også den nye modellen forbedret take-off dynamikk, der lettingen er mer realistisk ved både hastighet og altitude. Visualisering av denne simulasjonen er vedlagt som "JetSim2Visu.mp4", og fullstendig python kode er vedlagt som "PlaneModelNonLinear.py".

6 Etikk

Jagerfly innebærer mange etiske konsekvenser. For det første kan det argumenteres at det ikke burde eksistere våpen i det hele tatt fra ett etisk standpunkt, spesielt våpen som går langt utenfor det som trengs for å beskytte seg selv fra dyr og andre naturlige farer. Spesielt fra ett samfunnsmessigst perspektiv kan jagerfly medføre mange negative konsekvenser. Jagerfly og reguleringsteknologien bak den står like mye for å unngå fare, som det gjør å skape det; Jagerfly blir brukt både til bombeangrep selv, og får å stoppe fiendtlige bombefly eller møte ukjent luftfart for å utforske hva det er. Videre er jagerflyets manøvrerbarhet både for å unngå fiendtlige angrep, som missiler og kuller, like mye som for å få flyet i en posisjon til å angripe andre fly. Alt i alt er jagerfly stort sett en uetisk teknologi, men det kan likevel argumenteres at bruken av jagerfly for utforskning og air-patrol er en positiv konsekvens.

7 Konklusjon

I denne rapporten har jeg modellert et jagerfly i et ulineært tilstandsrom, som inkluderer en tilnærming av flyets aerodynamikk, både for luftmostand, demping i aksene, og løftekraft. Modellen ble brukt for å simulere takeoff, fulgt av en [Pugachevs Cobra manøver](#), og til slutt å falle ukontrollert gjennom luften. Dette ble også visualisert i 3D ved hjelp av Unity. Plott fra den endelige simuleringen vises i figur 10. I tillegg var flyets sensorer vurdert, der flyet har sensorer til å direkte observere alle tilstander. På lignende måte var også kontrollerbarhet testet ved lineariseringer av modellen ved flere punkter, der flyet ikke er fullt kontrollerbart når det har null fart, eller peker direkte oppover. Modellen er relativt presis og realistisk, men har noen mindre realistiske deler, som for eksempel at den faller veldig sakte grunnet en upresis tilnærming av vertikal luftmotstand.

References

- [1] Tonoy Chowdhury, Rewzana Linda, and Atul Garg. "Evolution of Flight Control System Fly-By-Light Control System". In: *International Journal of Emerging Technology and Advanced Engineering(IJETAE)* ; ISSN 2250-2459, ISO 9001:2008 Certified Journal Volume 3 (Dec. 2013).
- [2] Catalin Nae, Ilie Nicolin, and Nicolin Adrian. "Military Aircraft Flight Control". In: June 2022. ISBN: ISBN 978-1-80355-301-6. DOI: [10.5772/intechopen.105491](https://doi.org/10.5772/intechopen.105491).
- [3] Wolfram Research. *NonlinearStateSpaceModel*. 2014. URL: <https://reference.wolfram.com/language/ref/NonlinearStateSpaceModel.html>.
- [4] Wikipedia. *Jacobian matrix and determinant*. 2025. URL: https://en.wikipedia.org/wiki/Jacobian_matrix_and_determinant.